

complete list. Here, only sample methods are provided:

```
df = DataFrame(rand(7, 8))
```

```
df[:, Between(:x2, :x4)]
```

We have built a DataFrame with seven rows and eight columns with random values. By default, the column names are from *x1* to *x8*. In this example, we have provided the command to fetch columns between *x2* and *x4*. Here, the columns *x2*, *x3* and *x4* are fetched. The output is shown in Figure 4.

The column numbers can also be specified as given below:

```
df = DataFrame(rand(7, 8))
```

```
print(df[:, [3, 5]])
```

Here, columns 3 and 5 are displayed.

You can retrieve the specific rows and columns with the following syntax:

```
df = DataFrame(rand(7, 8))
```

```
df[[4, 6, 7], [:x2,:x4]]
```

Here, columns *x2* and *x4* are retrieved from rows 4, 6 and 7, as shown in Figure 5.

Grouping

groupby() is used to group values, as shown in the following example:

```
df = DataFrame(month=[1,2,1,2,3,1,3,2],
sales=[1500, 3000, 4500, 8500, 7000,
6000, 8900, 7500], v=rand(8))
groupby(df, :month)
```

The output with the values grouped by the 'month' column is shown in Figure 6.

During *Groupby*, sorting can be done, and missing values can also be handled (Figure 7).

```
groupby(df, :month, sort=true,
skipmissing=true)
```

7 rows × 3 columns

	x2	x3	x4
	Float64	Float64	Float64
1	0.746826	0.795659	0.309948
2	0.0486222	0.916439	0.262068
3	0.769891	0.0341501	0.574462
4	0.0968714	0.322072	0.851968
5	0.199382	0.927542	0.312306
6	0.327236	0.0499791	0.999087
7	0.448303	0.45826	0.247366

Figure 4: Fetching specific columns using *Between()*

```
df = DataFrame(rand(7, 8))
```

```
df[[4, 6, 7], [:x2,:x4]]
```

3 rows × 2 columns

	x2	x4
	Float64	Float64
1	0.288271	0.449127
2	0.805	0.327138
3	0.0173192	0.336449

Figure 5: Retrieving specific rows and columns

```
groupby(df, :month, sort=true, skipmissing=true)
```

GroupedDataFrame with 3 groups based on key: month

First Group (3 rows): month = 1

	month	sales	v
	Int64	Int64	Float64
1	1	1500	0.531029
2	1	4500	0.669483
3	1	6000	0.664792

⋮

Last Group (2 rows): month = 3

	month	sales	v
	Int64	Int64	Float64
1	3	7000	0.446277
2	3	8900	0.596681

Figure 6: Grouped DataFrame

GroupedDataFrame with 3 groups based on key: month

First Group (3 rows): month = 1

	month	sales	v
	Int64	Int64	Float64
1	1	1500	0.531029
2	1	4500	0.669483
3	1	6000	0.664792

⋮

Last Group (2 rows): month = 3

	month	sales	v
	Int64	Int64	Float64
1	3	7000	0.446277
2	3	8900	0.596681

Figure 7: *Groupby* with sort and missing values

As stated earlier, the objective of this article is to introduce the features of DataFrames.jl. For the complete enumeration of all the features, interested readers may refer to the official documentation. DataFrames.jl, with the lightning

execution speed of Julia, is a great toolkit if you are handling data-intensive applications. Try exploring DataFrames.jl further with a free course (<https://juliaacademy.com/p/introduction-to-dataframes-jl>) from the Julia academy. **END** 

References

- [1] http://juliadata.github.io/DataFrames.jl/stable/man/getting_started/#Getting-Started
- [2] <https://github.com/bkamins/Julia-DataFrames-Tutorial>
- [3] <https://juliaacademy.com/p/introduction-to-dataframes-jl>

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the School of Engineering and Technology, Pondicherry Central University. He has 14+ years of teaching and research experience in the academia and industry. His research interests include accessible computing, Web information retrieval and mobile computing.